4 Refactoring

The source code of an application isn't a static structure that is written once and then never touched again; rather, it's a dynamic entity that you have to deal with very often. In most cases, you read the source code and have to modify it much less frequently. This modification is often referred to as *refactoring*.

Refactoring is an important part of software development as it serves to improve the existing source code. There can be many reasons for refactoring, including cleaning up existing code that was created quickly, realizing that something has been learned in the course of developing the application or that application conventions have changed, making necessary modernizations because new versions of the libraries and frameworks have been released, and so on.

No matter what motivates you to refactor, it's always a critical and usually time-consuming intervention in your application. Luckily, tools are available to support you. Modern development environments can provide support for simple tasks such as renaming or extracting structures, but even they reach their limits at some point. This is precisely where AI tools can provide good support.

In most cases, refactoring isn't about creative and challenging tasks, but about adapting many code sections in the same way. AI tools really come into their own with routine tasks like these, as they can relieve you of repetitive tasks and thus reduce errors.

However, along with the advantages of using AI tools comes some risks. If you're aware of these risks, you can take countermeasures at an early stage to minimize them. In this chapter, you'll find out what these potential problems are and how you can deal with them.

4.1 Introduction to Refactoring

Before we dive into refactoring using AI tools, let's first take a look behind the scenes and find out exactly what refactoring is all about so that we can get the best possible help. *Refactoring* is the process of improving the internal structure of an application's source code, so you should separate the development of new features from refactoring. With refactoring, the focus is instead purely on the existing internal structure. The interfaces to other parts of the application or even to other systems remain the same.

4.1.1 Objectives of Refactoring

Refactoring should never be an end in itself. Instead, you should plan what you specifically want to improve. If you don't adhere to this, you can change the code, but you won't improve it. The aim of refactoring is to address one or more of the following aspects:

- **Improved readability**
  The structure of the source code gets designed in such a way that the code becomes easier to read and understand.
- **Increased maintainability**
  Refactoring adapts the structure of the code to make future changes and extensions easier.
- **Reduction of errors**
  Known sources of error can be minimized through targeted refactoring.
- **Increased efficiency**
  With source code that has grown over a longer period of time, inefficient structures creep in that slow down execution. Refactoring can improve these areas and thus increase the performance of the application.

Through continuous refactoring, you create a stable and flexible code base that enables you to react faster and more reliably to new requirements. Your application therefore remains future proof in the long term.